

TIME COMPLEXITY

Decidability of a problem merely indicates that the problem is computationally solvable *in principle*.

It may not be solvable *in practice* if the solution requires an inordinate amount of time or memory.

We shall introduce a way of measuring the time used to solve a problem.

We then show how to classify problems according to the amount of time required.

MEASURING TIME COMPLEXITY

Let $A = \{0^k 1^k \mid k \geq 0\}$.

How much time does a single-tape TM need to decide A ?

A single-tape TM M_1 for A works as follows:

1. Scan across the tape and *reject* if a 0 appears to the right of a 1.
2. Repeat Stage 3 if both 0s and 1s remain on the tape.
3. Scan across the tape, crossing off a single 0 and a single 1.
4. If no 0s or 1s remain on the tape, *accept*; otherwise, *reject*.

MEASURING TIME COMPLEXITY (CONT.)

We shall compute the running time of an algorithm purely as a function of the length of the string representing the input.

Let M be a deterministic TM that halts on all inputs.

The **running time** or **time complexity** of M is the function $f : \mathcal{N} \longrightarrow \mathcal{N}$, where $f(n)$ is the *maximum* number of steps that M uses on any input of length n .

If $f(n)$ is the running time of M , we say that M *runs in time* $f(n)$ or that M *is an* $f(n)$ *time Turing machine*.

ASYMPTOTIC ANALYSIS

The exact running time of an algorithm is a complex expression.

We seek to understand the running time of the algorithm when it is run on large inputs.

We do so by considering only the highest-order term of the expression of its running time (discarding the coefficient of that term and any lower-order terms).

For example, if $f(n) = 6n^3 + 2n^2 + 20n + 45$, we say that f is *asymptotically* at most n^3 .

The *asymptotic notation*, or *big-O notation*, for describing this relationship is $f(n) = O(n^3)$.

ASYMPTOTIC BOUNDS

Let $\mathcal{R}^+$ be the set of positive real numbers.

Let f and g be two functions $f, g : \mathcal{N} \longrightarrow \mathcal{R}^+$.

We say that $f(n) = O(g(n))$ if positive integers c and n_0 exist so that, for every integer $n \geq n_0$,

$$f(n) \leq cg(n).$$

When $f(n) = O(g(n))$, we say that $g(n)$ is an (asymptotic) *upper bound* for $f(n)$.

ASYMPTOTIC BOUNDS (CONT.)

Intuitively, $f(n) = O(g(n))$ means that f is less than or equal to g if we disregard differences up to a constant factor.

Big- O notation can appear in arithmetic expressions such as $O(n^2) + O(n)$ ($= O(n^2)$) and $2^{O(n)}$.

Big- O notation gives a way to say that one function is asymptotically *no more than* another.

ASYMPTOTIC BOUNDS (CONT.)

To say that one function is asymptotically *less than* another, we use small- o notation.

Let f and g be two functions $f, g : \mathcal{N} \longrightarrow \mathcal{R}^+$.

We say that $f(n) = o(g(n))$ if

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

For example, $\sqrt{n} = o(n)$ and $n \log n = o(n^2)$.

ANALYZING ALGORITHMS

Consider the single-tape TM M_1 for deciding $\{0^k1^k \mid k \geq 0\}$.

Stage 1 takes $2n (= O(n))$ steps: n steps to scan the input and another n steps to reposition the head at the left-hand end of the tape.

Each execution of Stage 3 takes $2n$ steps and at most $n/2$ such executions are required. So, Stages 2 and 3 take at most $(n/2)2n (= O(n^2))$ steps.

Stage 4 takes $n (= O(n))$ steps.

COMPLEXITY CLASSES

Let $t : \mathcal{N} \longrightarrow \mathcal{N}$ be a function.

Define the **time complexity class** $\text{TIME}(t(n))$ to be $\{L \mid L \text{ is a language decided by an } O(t(n)) \text{ time Turing machine}\}$.

$A (= \{0^k 1^k \mid k \geq 0\}) \in \text{TIME}(n^2)$, since M_1 decides A in time $O(n^2)$.

Is there a machine that decides A asymptotically faster?

In other words, is A in $\text{TIME}(t(n))$ for $t(n) = o(n^2)$?

COMPLEXITY CLASSES (CONT.)

Below is a faster single-tape TM for deciding $A (= \{0^k 1^k \mid k \geq 0\})$.

$M_2 =$ “On input string w :

1. Same as Stage 1 of M_1 .
2. Repeat Stages 3 and 4 if both 0s and 1s remain on the tape.
3. If the total number of 0s and 1s remaining is odd, *reject*.
4. Cross off every other 0 and then every other 1.
5. If no 0s or 1s remain on the tape, *accept*; otherwise, *reject*.”

The running time of M_2 is $O(n \log n)$ and hence $A \in \text{TIME}(n \log n)$.

COMPLEXITY CLASSES (CONT.)

Below is an even faster TM, which has *two* tapes, for deciding $A (= \{0^k 1^k \mid k \geq 0\})$.

$M_3 =$ “On input string w :

1. Same as Stage 1 of M_1 .
2. Copy the 0s on Tape 1 onto Tape 2.
3. Scan across the 1s on Tape 1 until the end of the input, crossing off a 0 on Tape 2 for each 1. If there are not enough 0s, *reject*.
4. If all the 0s have now been crossed off, *accept*; otherwise, *reject*.”

COMPLEXITY CLASSES (CONT.)

The running time of M_3 is $O(n)$.

This indicates that the complexity of A depends on the model of computation selected.

COMPLEXITY RELATIONSHIPS AMONG MODELS

Let $t(n)$ be a function, where $t(n) \geq n$. Then every $t(n)$ time multitape Turing machine has an equivalent $O(t^2(n))$ time single-tape Turing machine.

Let M be a k -tape TM running in $t(n)$ time.

A single-tape TM S simulating M requires $O(t(n))$ tape cells to store the current contents of M 's tapes and the respective head positions.

It takes $O(t(n))$ time for S to simulate each of M 's $t(n)$ steps.

So, the running time of S is $t(n) \times O(t(n)) = O(t^2(n))$.

COMPLEXITY RELATIONSHIPS AMONG MODELS (CONT.)

The running time of a nondeterministic TM N is the function $f : \mathcal{N} \longrightarrow \mathcal{N}$, where $f(n)$ is the *maximum* number of steps that N uses on *any* branch of its computation on any input of length n .

Let $t(n)$ be a function, where $t(n) \geq n$. Then every $t(n)$ time nondeterministic single-tape Turing machine has an equivalent $2^{O(t(n))}$ time deterministic single-tape Turing machine.

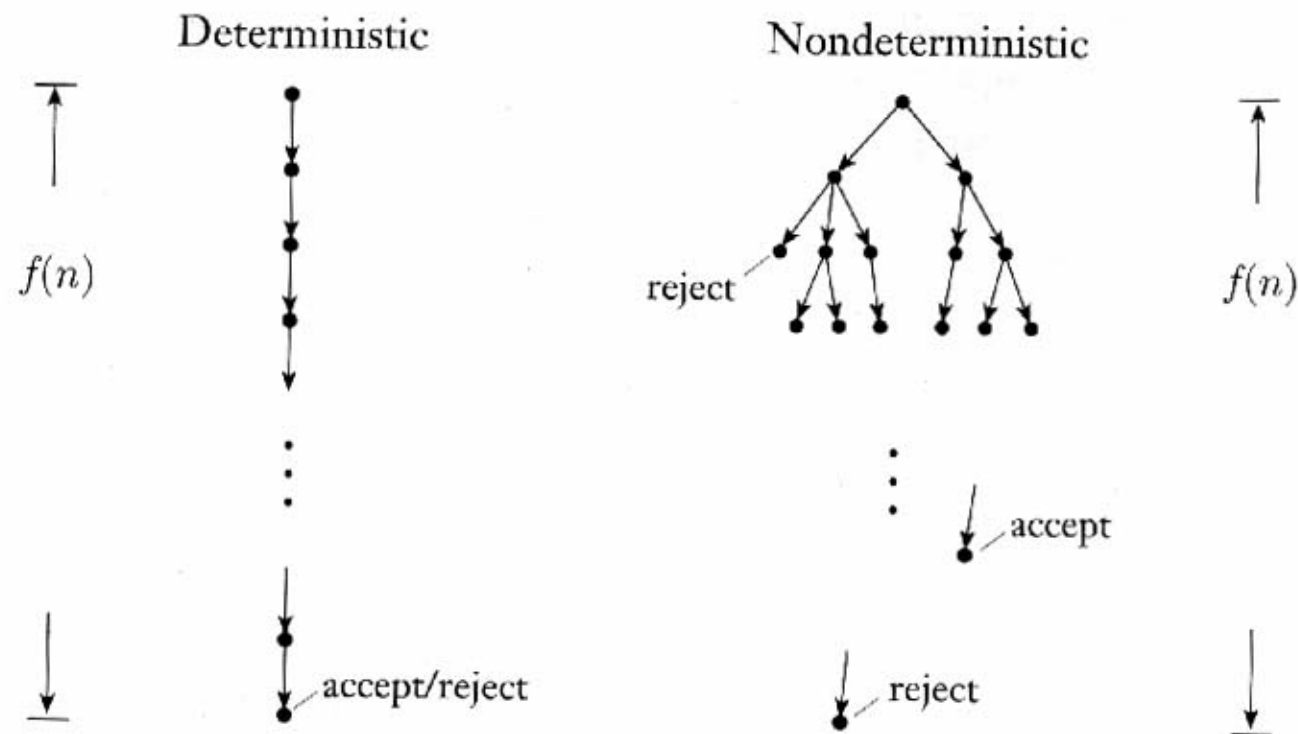


FIGURE 7.1
Measuring deterministic and nondeterministic time

COMPLEXITY RELATIONSHIPS AMONG MODELS (CONT.)

Every branch of N 's computation tree has a length of at most $t(n)$.

The total number of nodes in the tree is $O(b^{t(n)})$, where b is the maximum number of legal choices given by N 's transition function.

The running time of a simulating deterministic 3-tape TM is $O(t(n)) \times O(b^{t(n)}) = 2^{O(t(n))}$.

The running time of a simulating deterministic single-tape TM is $(2^{O(t(n))})^2 = 2^{O(2t(n))} = 2^{O(t(n))}$.

POLYNOMIAL TIME

For our purposes, *polynomial differences* in running time are considered to be small, whereas *exponential differences* are considered to be large.

Exponential time algorithms typically arise when we solve problems by searching through a space of solutions, called *brute-force search*.

All “reasonable” deterministic computational models are *polynomially equivalent*, i.e., any one of them can simulate another with a polynomial increase in running time.

We shall focus on aspects of time complexity theory that are unaffected by polynomial differences in running time.

THE CLASS P

P is the class of languages that are decidable in *polynomial* time on a *deterministic* single-tape Turing machine. In other words,

$$P = \bigcup_k \text{TIME}(n^k)$$

1. P is invariant for all models of computing that are polynomially equivalent to the deterministic single-tape Turing machine.
2. P roughly corresponds to the class of problems that are realistically solvable on a computer.

ANALYZING ALGORITHMS FOR P PROBLEMS

Suppose that we have given a high-level description of a polynomial time algorithm with stages. To analyze the algorithm,

1. we first give a polynomial upper bound on the number of stages that the algorithm uses, and
2. we then show that the individual stages can be implemented in polynomial time on a reasonable deterministic model.

A “reasonable” encoding method for problems should be used, which allows for polynomial time encoding and decoding of objects into natural internal representation or into other reasonable encodings.

PROBLEMS IN P

$PATH = \{\langle G, s, t \rangle \mid G \text{ is a directed graph that has a directed path from } s \text{ to } t\}.$

$PATH \in P.$

$M =$ “On input $\langle G, s, t \rangle$:

1. Place a mark on node s .
2. Repeat Stage 3 until no additional nodes are marked.
3. Scan all the edges of G . If an edge (a, b) is found going from a marked node a to an unmarked node b , mark node b .
4. If t is marked, *accept*; otherwise, *reject*.”

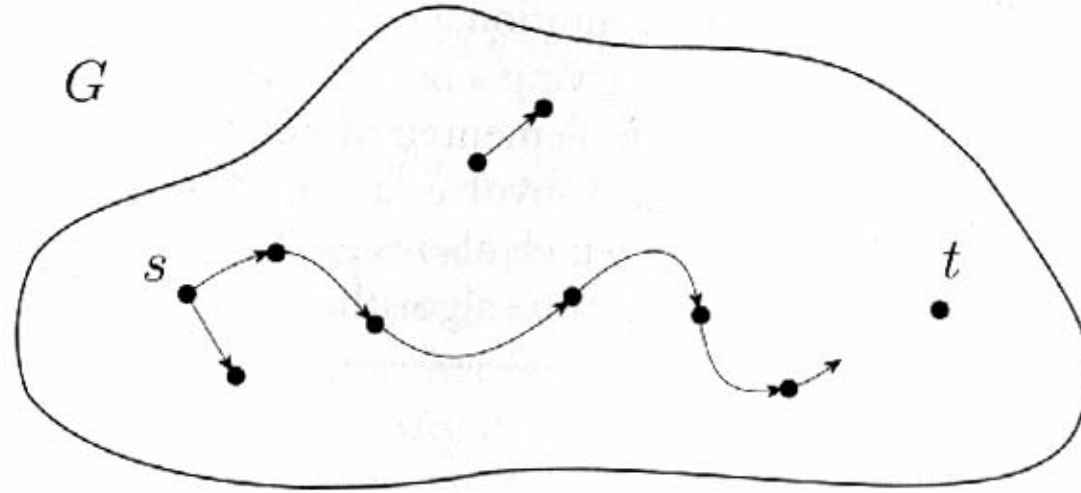


FIGURE 7.2

The *PATH* problem: Is there a path from s to t ?

PROBLEMS IN P (CONT.)

$RELPRIME = \{\langle x, y \rangle \mid x \text{ and } y \text{ are relatively prime}\}.$

$RELPRIME \in P.$

$E =$ “On input $\langle x, y \rangle$:

1. Repeat Stages 2 and 3 until $y = 0$.
2. Assign $x \leftarrow x \bmod y$.
3. Exchange x and y .
4. Output x .”

PROBLEMS IN P (CONT.)

$R =$ “On input $\langle x, y \rangle$:

1. Run E on $\langle x, y \rangle$.
2. If E 's output is 1, *accept*; otherwise, *reject*.”

PROBLEMS IN P (CONT.)

Every context-free language belongs to P .

$D =$ "On input $w = w_1w_2 \cdots w_n$,

1. If $w = \varepsilon$ and $S \rightarrow \varepsilon$ is a rule, *accept*.
2. For $i = 1$ to n ,
3. For each variable A ,
4. Is $A \rightarrow b$, where $b = w_i$, a rule?
5. If yes, add A to $table(i, i)$.
6. For $l = 2$ to n ,
7. For $i = 1$ to $n - l + 1$,
8. Let $j = i + l - 1$,
9. For $k = i$ to $j - 1$,
10. For each rule $A \rightarrow BC$,
11. If $B \in table(i, k)$ and $C \in table(k + 1, j)$,
 then put A in $table(i, j)$.
12. If $S \in table(1, n)$, *accept*; otherwise, *reject*."

THE HAMILTONIAN PATH PROBLEM

A *Hamiltonian path* in a directed graph is a directed path that goes through each node exactly once.

$HAMPATH = \{\langle G, s, t \rangle \mid G \text{ is a directed graph with a Hamiltonian path from } s \text{ to } t\}$.

We can easily obtain an exponential time algorithm for *HAMPATH*.

No one knows whether *HAMPATH* is solvable in polynomial time.

However, *verifying* the existence of a Hamiltonian path may be much easier than *determining* its existence.

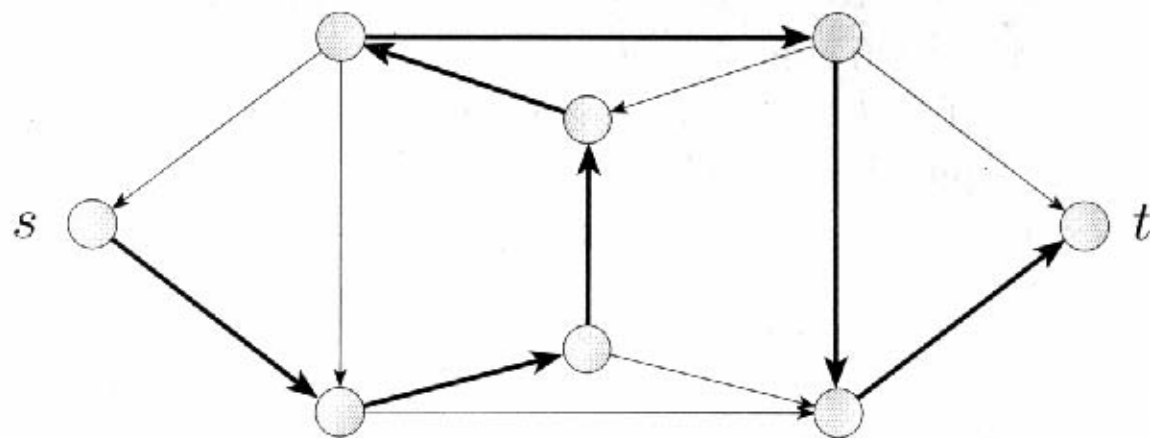


FIGURE 7.3

A Hamiltonian path goes through every node exactly once

THE CLASS NP

A **verifier** for a language A is an algorithm V , where

$$A = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}.$$

The information represented by the symbol c is called a *certificate*, or *proof*, of membership in A .

A *polynomial time verifier* runs in polynomial time in the length of w .

NP is the class of *polynomially verifiable* languages, i.e., languages that have polynomial time verifiers.

THE CLASS NP (CONT.)

A language is in NP iff it is decided by some nondeterministic polynomial time Turing machine.

Let V be a verifier for $A \in \text{NP}$ that runs in time n^k . Construct a decider N for A as follows.

$N =$ “On input w of length n :

1. Nondeterministically select string c of length n^k .
2. Run V on input $\langle w, c \rangle$.
3. If V accepts, *accept*; otherwise, *reject*.”

THE CLASS NP (CONT.)

Let N be a nondeterministic decider for a language A that runs in time n^k . Construct a verifier V for A as follows.

$V =$ "On input $\langle w, c \rangle$:

1. Simulate N on input w , treating each symbol of c as a description of the nondeterministic choice to make at each step.
2. If this branch of N 's computation accepts, *accept*; otherwise, *reject*."

THE CLASS NP (CONT.)

$\text{NTIME}(t(n)) = \{L \mid L \text{ is a language decided by an } O(t(n)) \text{ time nondeterministic Turing machine}\}.$

$$\text{NP} = \bigcup_k \text{NTIME}(n^k).$$

ANALYZING ALGORITHMS FOR NP PROBLEMS

The class NP is insensitive to the choice of reasonable nondeterministic computational model.

Like in the deterministic case, we use a high-level description to present a nondeterministic polynomial time algorithm.

1. Each stage of a nondeterministic polynomial time algorithm must have an obvious implementation in polynomial on a reasonable nondeterministic model.
2. Every branch of its computation tree uses at most polynomially many stages.

PROBLEMS IN NP

A *clique* in an undirected graph is a subgraph, wherein every two nodes are connected by an edge.

A *k-clique* is a clique that contains k nodes.

$CLIQUE = \{\langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique}\}.$

$CLIQUE$ is in NP.

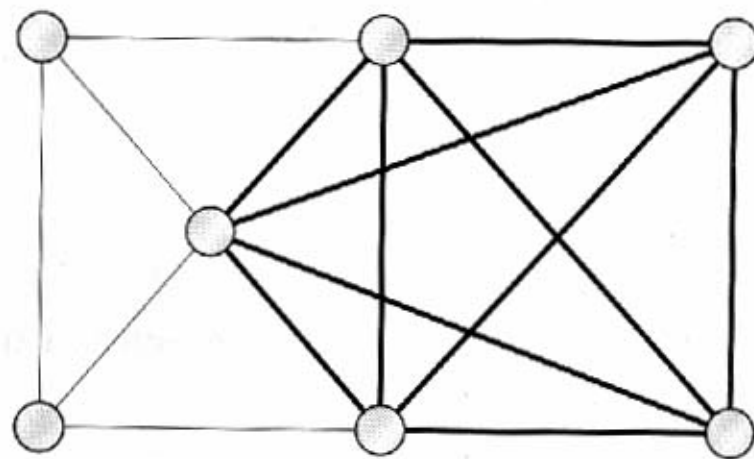


FIGURE 7.4
A graph with a 5-clique

PROBLEMS IN NP (CONT.)

$V =$ “On input $\langle\langle G, k \rangle, c \rangle$:

1. Test whether c is a set of k nodes in G .
2. Test whether G contains all edges connecting nodes in c .
3. If both pass, *accept*; otherwise, *reject*.”

$N =$ “On input $\langle G, k \rangle$:

1. Nondeterministically select a subset c of k nodes in G .
2. Test whether G contains all edges connecting nodes in c .
3. If yes, *accept*; otherwise, *reject*.”

PROBLEMS IN NP (CONT.)

$SUBSET_SUM = \{\langle S, t \rangle \mid S = \{x_1, \dots, x_k\} \text{ and for some } \{y_1, \dots, y_l\} \subseteq S, \text{ we have } \sum y_i = t\}.$

$SUBSET_SUM$ is in NP.

$V =$ “On input $\langle \langle S, t \rangle, c \rangle$:

1. Test whether c is a collection of numbers that sum to t .
2. Test whether S contains the numbers in c .
3. If both pass, *accept*; otherwise, *reject*.”

PROBLEMS IN NP (CONT.)

$N =$ “On input $\langle S, t \rangle$:

1. Nondeterministically select a subset c of the numbers in S .
2. Test whether c is a collection of numbers that sum to t .
3. If yes, *accept*; otherwise, *reject*.”

THE CLASS co-NP

The complements of *CLIQUE* and *SUBSET_SUM*, namely $\overline{CLIQUE}$ and $\overline{SUBSET_SUM}$, are not obviously members of NP.

Verifying that something is *not* present seems to be more difficult than verifying that it *is* present.

The complexity class co-NP contains the languages that are complements of languages in NP.

We do not know whether co-NP is different from NP.

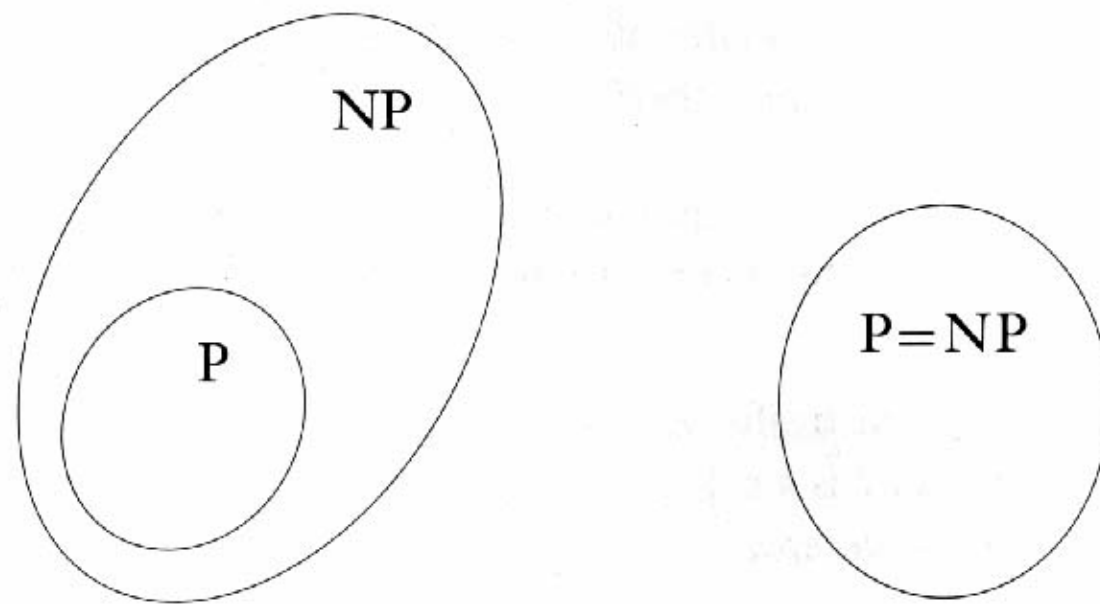


FIGURE 7.5

One of these two possibilities is correct

NP-COMPLETENESS

The complexity of certain problems in NP is related to that of the entire class [Cook and Levin].

If a polynomial time algorithm exists for any of the problems, all problems in NP would be polynomial time solvable.

These problems are called **NP-complete**.

$SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}.$

$SAT \in P \text{ iff } P = NP.$

POLYNOMIAL TIME REDUCIBILITY

When problem A is *efficiently* reducible to problem B , an efficient solution to B can be used to solve A efficiently.

A function $f : \Sigma^* \longrightarrow \Sigma^*$ is a **polynomial time computable function** if some polynomial time Turing machine M , on every input w , halts with just $f(w)$ on its tape.

Language A is **polynomial time mapping reducible** (polynomial time reducible) to language B , written $A \leq_p B$, if there is a polynomial time computable function $f : \Sigma^* \longrightarrow \Sigma^*$, where for every w ,

$$w \in A \iff f(w) \in B.$$

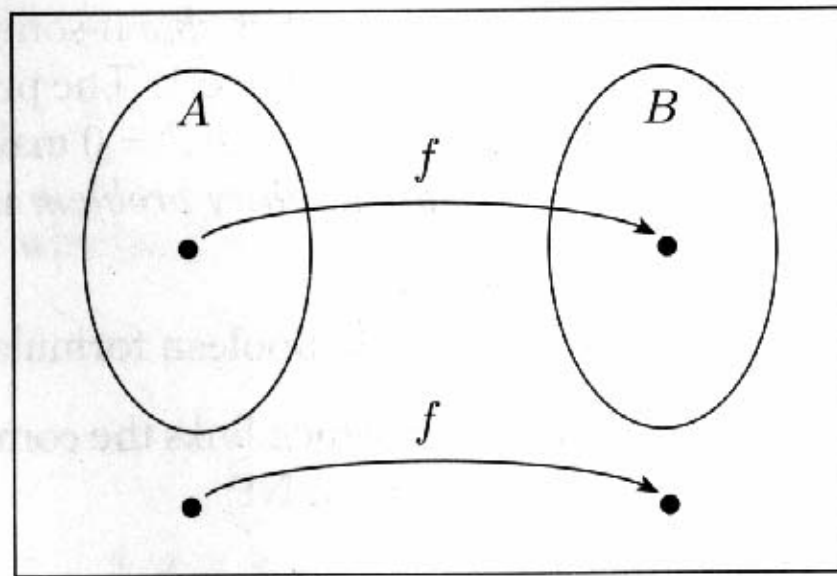


FIGURE 7.6

Polynomial time function f reducing A to B

POLYNOMIAL TIME REDUCIBILITY (CONT.)

If $A \leq_p B$ and $B \in P$, then $A \in P$.
--

Let M be the polynomial time algorithm deciding B and f be the polynomial time reduction from A to B .

$N =$ “On input w :

1. Compute $f(w)$.
2. Run M on input $f(w)$ and output whatever M outputs.”

EXAMPLE POLYNOMIAL TIME REDUCIBILITY

A Boolean formula is in *conjunctive normal form*, called a CNF-formula, if it comprises several clauses connected with $\wedge$ s, as in

$$(x_1 \vee \overline{x_2} \vee \overline{x_3} \vee x_4) \wedge (x_3 \vee \overline{x_5} \vee x_6) \wedge (x_3 \vee \overline{x_6})$$

It is a 3CNF-formula if all the clauses have three literals, as in

$$(x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_5} \vee x_6) \wedge (x_3 \vee \overline{x_6} \vee x_4) \wedge (x_4 \vee x_5 \vee x_6)$$

$3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3CNF-formula}\}.$

$3SAT$ is polynomial time reducible to $CLIQUE$.

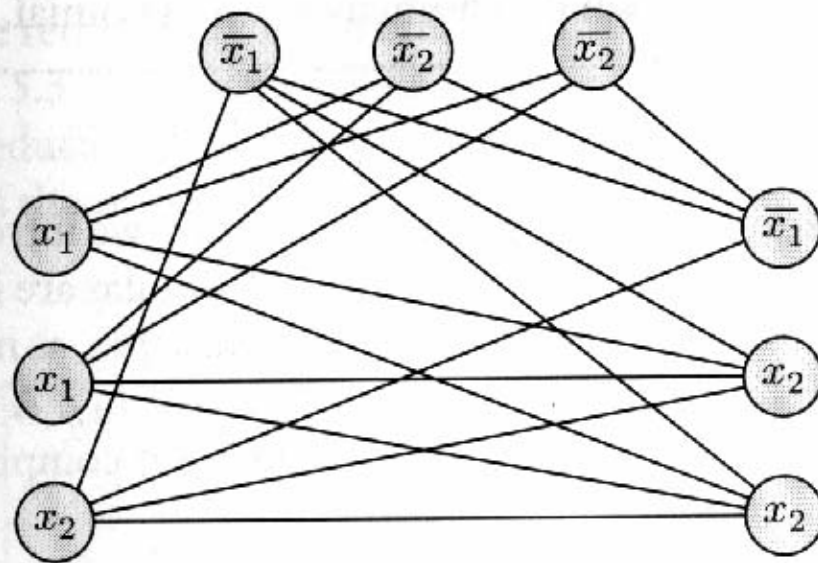


FIGURE 7.7

The graph that the reduction produces from

$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$

NP-COMPLETENESS

A language B is **NP-complete** if it satisfies two conditions:

1. B is in NP, and
2. every A in NP is polynomial time reducible to B .

If B is NP-complete and $B \in P$, then $P = NP$.

If B is NP-complete and $B \leq_p C$ for some $C \in NP$, then C is NP-complete.

THE COOK-LEVIN THEOREM

<i>SAT</i> is NP-complete.

SAT is in NP, as a nondeterministic polynomial time TM can guess an assignment to a given formula ϕ and accept if the assignment satisfies ϕ .

We next construct a polynomial time reduction for each language A in NP to *SAT*.

The reduction takes a string w and produces a Boolean formula ϕ that simulates the NP machine N for A on input w .

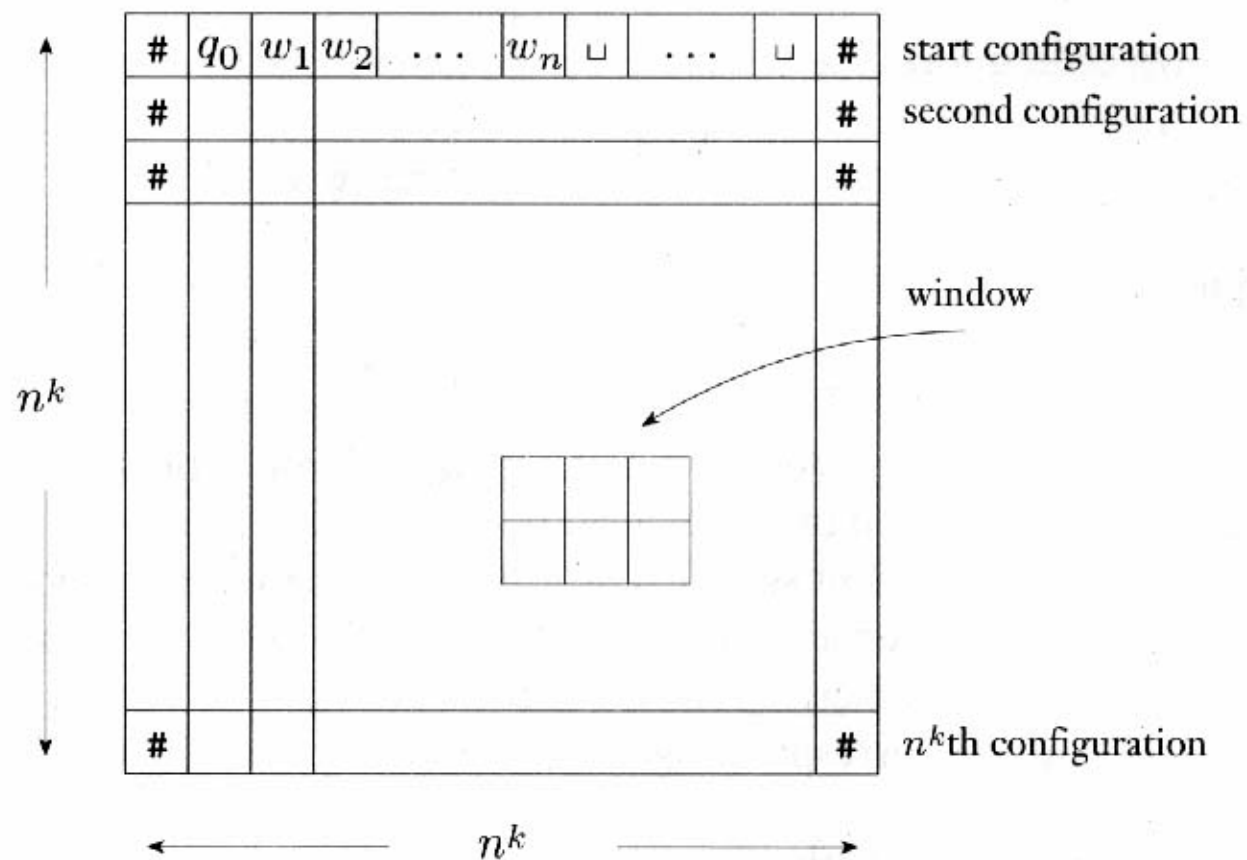


FIGURE 7.8

A tableau is an $n^k \times n^k$ table of configurations

THE COOK-LEVIN THEOREM (CONT.)

If N accepts, ϕ has a satisfying assignment that corresponds to the accepting computation.

If N rejects, no assignment satisfies ϕ .

Let $C = Q \cup \Gamma \cup \{\#\}$.

Construct ϕ as $\phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}}$, where ...

THE COOK-LEVIN THEOREM (CONT.)

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s, t \in C, s \neq t} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right] .$$

$$\begin{aligned} \phi_{\text{start}} = & x_{1,1,\#} \wedge x_{1,2,q_0} \wedge \\ & x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \cdots \wedge x_{1,n+2,w_n} \wedge \\ & x_{1,n+3,\sqcup} \wedge \cdots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#} . \end{aligned}$$

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}} .$$

$$\phi_{\text{move}} = \bigwedge_{1 \leq i \leq (n^k-1), 2 \leq j \leq (n^k-1)} (\text{window } (i, j) \text{ is legal}) .$$

Assume that $\delta(q_1, a) = \{(q_1, b, R)\}$ and $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$.

(a)

a	q_1	b
q_2	a	c

(b)

a	q_1	b
a	a	q_2

(c)

a	a	q_1
a	a	b

(d)

#	b	a
#	b	a

(e)

a	b	a
a	b	q_2

(f)

b	b	b
c	b	b

FIGURE 7.9
Examples of legal windows

Assume that $\delta(q_1, a) = \{(q_1, b, R)\}$ and $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$.

(a)

a	b	a
a	a	a

(b)

a	q_1	b
q_1	a	a

(c)

b	q_1	b
q_2	b	q_2

FIGURE 7.10
Examples of illegal windows

THE COOK-LEVIN THEOREM (CONT.)

The condition “window (i, j) is legal” can be expressed as

$$\bigvee_{a_1, \dots, a_6} \text{legal} \left(x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge \right. \\ \left. x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6} \right)$$

NP-COMPLETE PROBLEMS

$3SAT$ is NP-complete.

The proof of the Cook-Levin theorem can be modified so that the Boolean formula involved is in conjunctive normal form.

A CNF-formula can be converted in polynomial time to a 3CNF-formula (with a length polynomially bounded in the length of the CNF-formula).

If a clause contains l literals $(a_1 \vee a_2 \vee \cdots \vee a_l)$, we can replace it with the $l - 2$ clauses

$$(a_1 \vee a_2 \vee z_1) \wedge (\overline{z_1} \vee a_3 \vee z_2) \wedge (\overline{z_2} \vee a_4 \vee z_3) \wedge \cdots \wedge (\overline{z_{l-4}} \vee a_{i-1} \vee z_{l-3}) \wedge (\overline{z_{l-3}} \vee a_{i-1} \vee a_l)$$

NP-COMPLETE PROBLEMS (CONT.)

CLIQUE is NP-complete.

CLIQUE is in NP and $3SAT \leq_p CLIQUE$.

NP-COMPLETE PROBLEMS (CONT.)

A *vertex cover* of an undirected graph G is a subset of the nodes where every edge of G touches one of those nodes.

$VERTEX_COVER = \{\langle G, k \rangle \mid G \text{ is an undirected graph that has a } k\text{-node vertex cover}\}.$

$VERTEX_COVER$ is NP-complete.

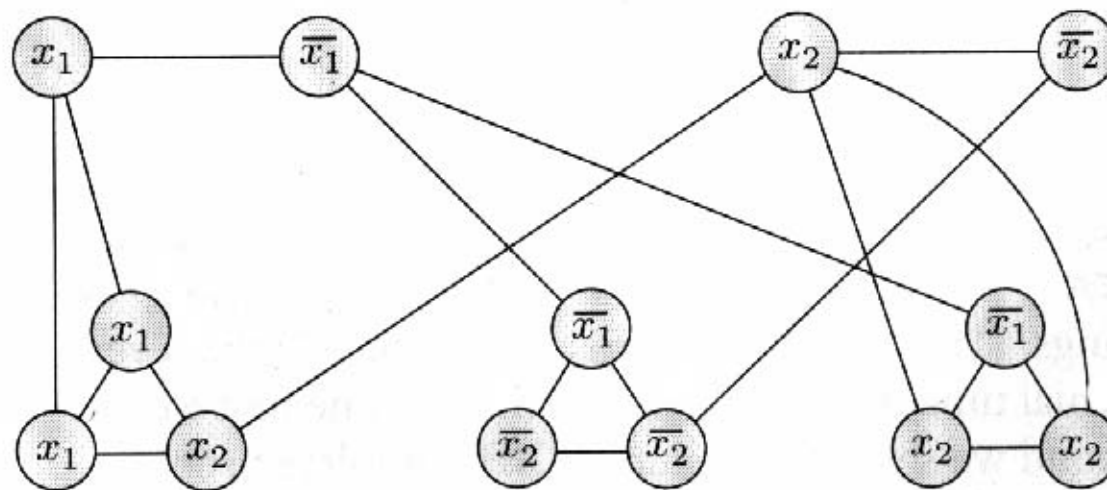


FIGURE 7.11

The graph that the reduction produces from

$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$

NP-COMPLETE PROBLEMS (CONT.)

HAMPATH is NP-complete.

We show that $3SAT \leq_P HAMPATH$.

NP-COMPLETE PROBLEMS (CONT.)

Let $UHAMPATH$ be the undirected version of the Hamiltonian path problem $HAMPATH$.

$UHAMPATH$ is NP-complete.

An input $\langle G, s, t \rangle$ for $HAMPATH$ is mapped to $\langle G', s', t' \rangle$ for $UHAMPATH$ as follows.

Each node u of G , except for s and t , is replaced by a triple of nodes u^{in} , u^{mid} , and u^{out} in G' .

Nodes s and t are replaced by node $s^{\text{out}} = s'$ and $t^{\text{in}} = t'$.

Edges connect u^{mid} with u^{in} and u^{out} .

An edge connects u^{out} and v^{in} if (u, v) is an edge of G .

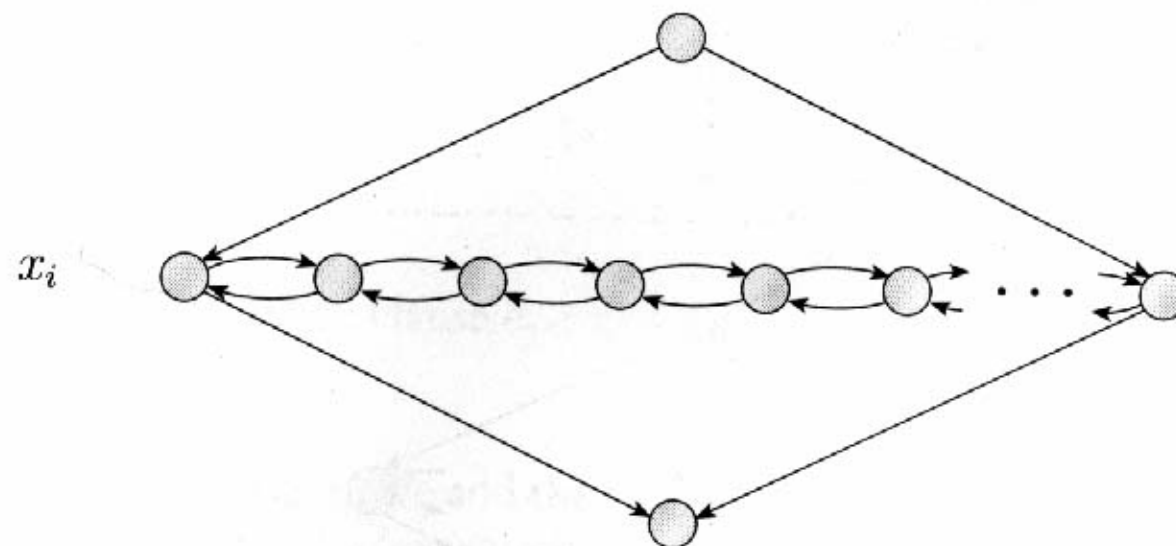


FIGURE 7.12
Representing the variable x_i as a diamond structure



FIGURE 7.13

Representing the clause c_j as a node

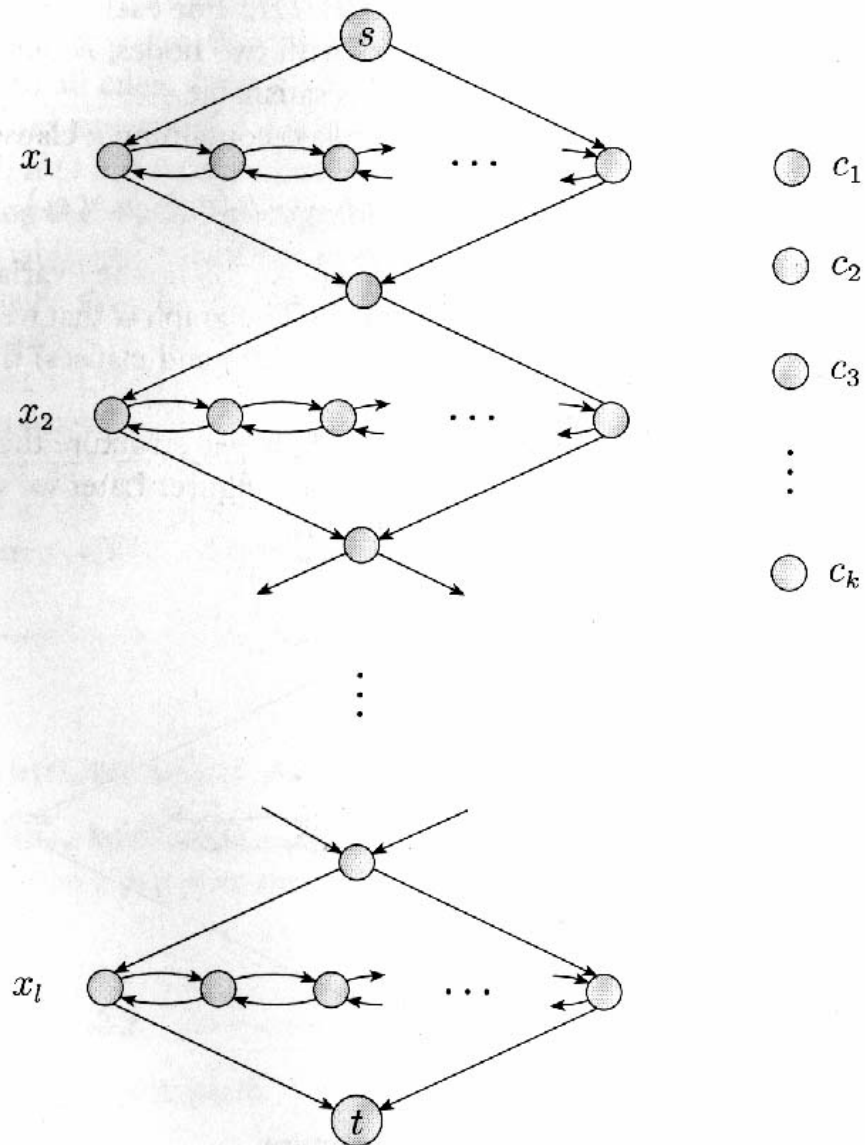


FIGURE 7.14
The high-level structure of G

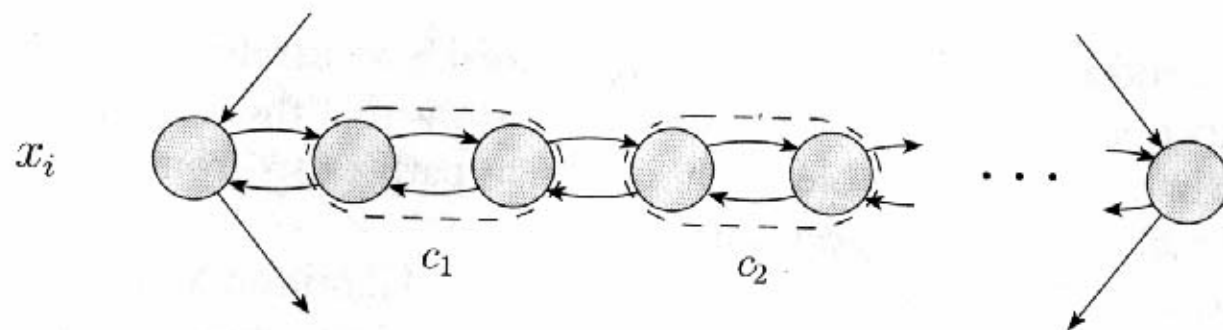


FIGURE 7.15
The horizontal nodes in a diamond structure

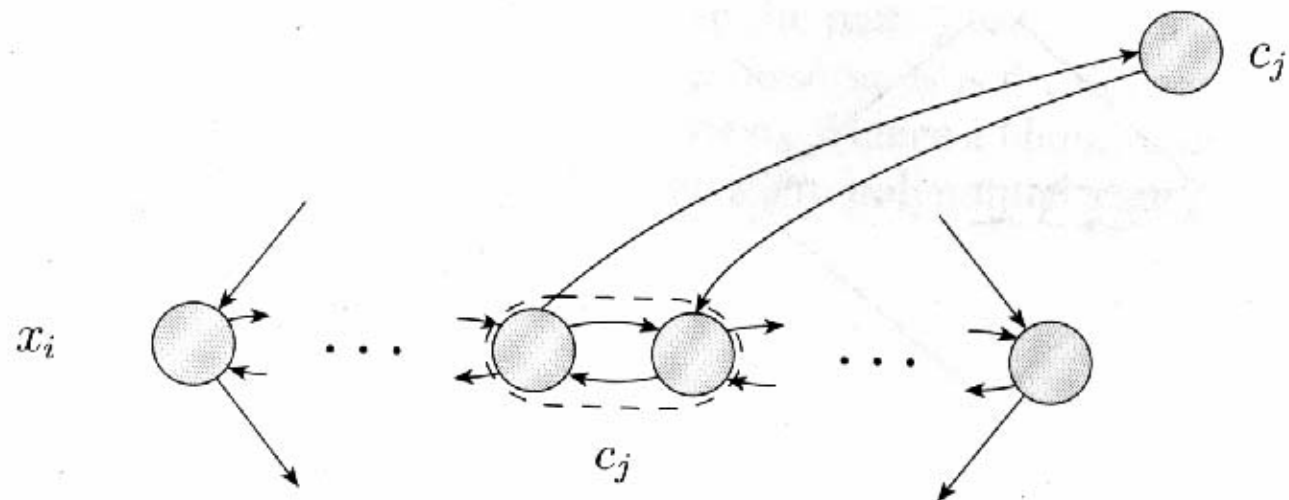


FIGURE 7.16

The additional edges when clause c_j contains x_i

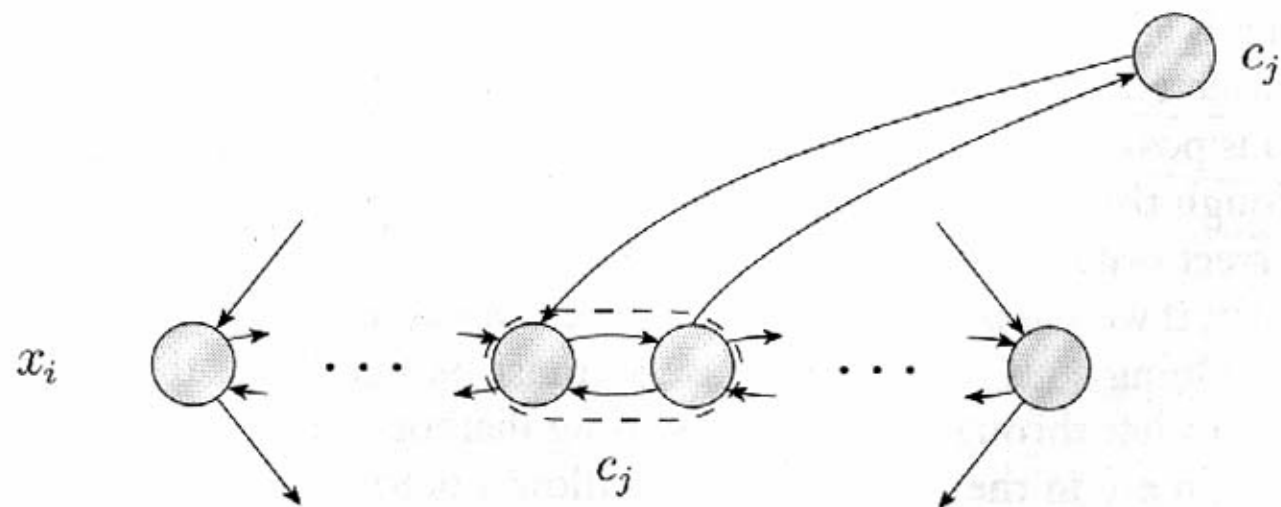


FIGURE 7.17

The additional edges when clause c_j contains $\overline{x_i}$

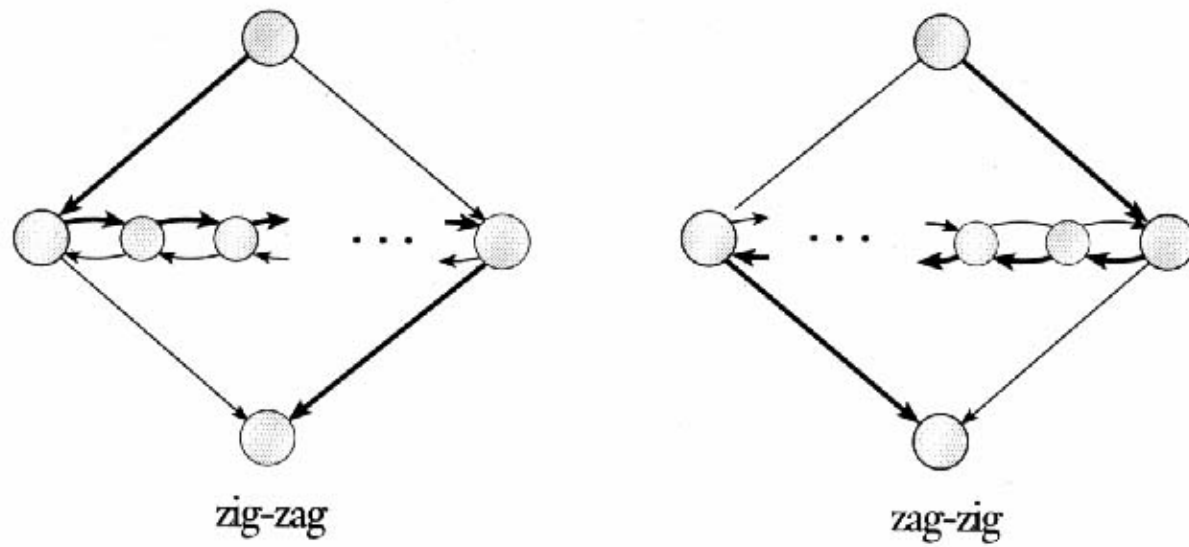


FIGURE 7.18

Zig-zagging and zag-zigging through a diamond, as determined by the satisfying assignment

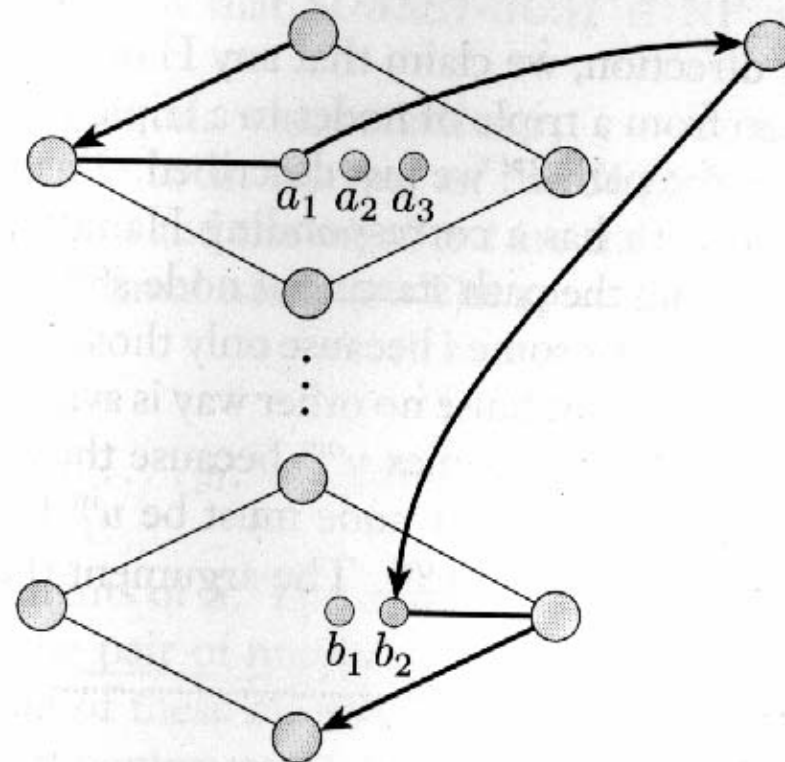


FIGURE 7.19

This situation cannot occur

NP-COMPLETE PROBLEMS (CONT.)

$SUBSET_SUM = \{\langle S, t \rangle \mid S = \{x_1, \dots, x_k\} \text{ and for some } \{y_1, \dots, y_l\} \subseteq S, \text{ we have } \sum y_i = t\}.$

$SUBSET_SUM$ is NP-complete.

We show that $3SAT \leq_P SUBSET_SUM$.